

CS 415

Software Engineering

Project Milestone 0 Proposal

StudyFlow

Smart Academic Workload and Deadline Management System

Team Members

Eunjoo Jung

ejung7@crimson.ua.edu

Errin James

emjames1@crimson.ua.edu

Shared GitHub Repository

<https://github.com/hihi0305/studyflow>

Submission Date

02 September 2026

1. Executive Summary

StudyFlow is a web-based academic planning application designed to help college students organize courses, assignments, exams, and projects in one centralized location. College students often manage multiple deadlines across different courses, and traditional task lists generally show what needs to be completed without helping students determine which tasks require the most immediate attention.

StudyFlow addresses this problem by combining academic task management with a dashboard that highlights tasks based on their Task Attention Level. Students will be able to create courses, add academic tasks, specify deadlines, estimate the amount of work required, indicate user-defined task priority, and track completion progress. The system will use this information to help students identify tasks that may require immediate attention.

The initial version of StudyFlow will focus on reliable academic task organization and a clear dashboard experience. Later milestones will introduce Task Attention Level calculations, filtering, testing, security improvements, deployment, and other production-quality software engineering practices.

2. Target Users & Personas

2.1 Target Users

The primary users of StudyFlow are college and university students who manage multiple courses and overlapping academic deadlines.

Secondary users may include part-time students, working students, online students, and students participating in multiple long-term academic projects. StudyFlow is particularly intended for students who need more information than a traditional calendar or basic to-do list provides when deciding what academic work should receive their attention first.

2.2 Persona 1 Traditional College Student

- Name: Maya
- Age: 20
- Role: Full-time undergraduate student

Maya is enrolled in four upper-level courses and regularly manages homework assignments, programming projects, exams, and group assignments. She often has overlapping deadlines and struggles to decide which task should receive her attention first. She currently keeps track of academic work across different places, such as course websites, calendars, and personal notes.

- Needs
 - One centralized place to manage academic tasks and deadlines
 - A clear overview of upcoming work
 - A way to identify urgent tasks quickly
- Goals
 - Avoid missing assignments and deadlines
 - Start large projects earlier
 - Balance her workload efficiently across multiple courses
- Constraints
 - Limited study time
 - Multiple overlapping deadlines
 - Heavy workloads from upper-level courses
 - Difficulty prioritizing tasks with different workload requirements

2.3 Persona 2 Working Student

- Name: Miguel
- Age: 21
- Role: Part-time student and part-time employee

Miguel is working a part-time job to help pay for college. He balances classes, assignments, and work shifts, and his available study time often changes depending on his work schedule. Because of this, simply knowing an assignment's due date is not always enough for him to decide what to work on first.

- Needs
 - A tool that clearly shows which tasks require the most immediate attention
 - Estimated workload information for upcoming assignments
 - Progress tracking for larger tasks and projects
- Goals

- Finish course assignments on time
 - Use limited study time efficiently
 - Avoid starting major assignments too late
- Constraints
 - Limited and irregular study time
 - Work obligations outside of class
 - Limited ability to study for long periods
 - Difficulty balancing academic responsibilities with changing work shifts

3. Proposed Application

3.1 Purpose

The purpose of StudyFlow is to provide students with a centralized academic workspace that combines course organization, task management, progress tracking, and workload awareness. Based on user-provided information such as deadlines, estimated work hours, user-defined priority, and progress, StudyFlow will help students identify which tasks require the most immediate attention and manage their academic workload more efficiently.

Unlike a basic to-do application, StudyFlow will consider multiple task characteristics rather than relying only on due dates when determining each task's Task Attention Level.

3.2 Expected User Experience

StudyFlow should provide a simple and intuitive experience where students can quickly add courses and academic tasks, view their workload from a centralized dashboard, update task progress, and easily identify which tasks require the most immediate attention.

3.3 Major Features

The proposed application will include the following major features:

- User Registration and Authentication
- Course Creation and Management
- Academic Task Creation and Management

- Centralized Academic Dashboard
- Task Attention Level Calculation
- Search and Filtering
- Responsive Web Interface

Later versions may include a calendar or timeline view, reminder notifications, academic workload analytics, study-plan suggestions, and AI-assisted task breakdown.

3.4 Intended Platform

StudyFlow will initially be developed as a responsive web application. Users will access the system through a modern desktop or mobile web browser.

The initial architecture will use a client-server design with a browser-based frontend, REST API backend, and relational database.

3.5 Typical End-to-end Workflow

- The user creates an account.
- The user logs in and is directed to their centralized dashboard.
- The user creates one or more courses.
- The user creates an academic task and assigns it to one of their existing courses.
- The user enters the task details, including:
 - Task title
 - Due date
 - Task type
 - User-defined priority
 - Estimated work hours
 - Current progress percentage
- StudyFlow stores the task, determines its Task Attention Level based on the available task information, and displays it on the user's dashboard.
- The user views active tasks and their related details from the dashboard.
- As work is completed, the user updates the task's progress from 0-100% and changes its status when appropriate.
- The user can edit or delete tasks as needed.
- The user can edit or delete courses as needed.
- When a task is finished, the user marks it as Completed.

4. MVP Scope

4.1 Required MVP Features

- User registration
- User login and logout
- Course creation
- Course editing and deletion
- Academic task creation
- Academic task editing and deletion
- Task type selection
- Due-date entry
- User-defined priority selection
- Estimated work-hour entry
- Task progress/status tracking
- Basic dashboard
- Persistent database storage
- User-specific data access

4.2 Features Excluded From the MVP

- AI-generated study plans
- Automatic syllabus parsing
- Calendar synchronization
- Push notifications
- Machine-learning prediction
- Social or collaborative features
- Native mobile applications
- Advanced analytics

These features are outside the initial scope because the team intends to prioritize a stable end-to-end core application before introducing advanced functionality.

4.3 Possible Later Features

- Task Attention Level calculation
- Workload-based task recommendations
- Calendar or timeline view
- Advanced search and filtering

- Academic workload analytics
- AI-assisted task decomposition
- Reminder notifications

5. Initial Requirements

5.1 Prioritized User Stories

ID	User Story	Priority	Initial Estimate
US-01	As a new user, I want to create an account so that I can securely access my academic information.	High	3
US-02	As a registered user, I want to log in and log out securely so that my academic data remains private.	High	5
US-03	As a student, I want to create and manage courses so that I can organize tasks by course.	High	3
US-04	As a student, I want to create academic tasks so that I can track assignments, exams, and projects.	High	5
US-05	As a student, I want to edit and delete tasks so that my academic information remains accurate.	High	3
US-06	As a student, I want to view my active tasks on a dashboard so that I can quickly understand my workload.	High	5
US-07	As a student, I want to record task progress so that I know how much work remains.	High	3
US-08	As a student, I want StudyFlow to assign Task Attention Levels to my active tasks so that I can decide what to work on first.	Medium	5

US-09	As a student, I want to search and filter tasks so that I can quickly find relevant academic work.	Medium	3
US-10	As a student, I want to view workload information so that I can better plan upcoming study time.	Medium	5
US-11	As a student, I want a calendar or timeline view so that I can understand upcoming deadlines visually.	Low	5
US-12	As a student, I want StudyFlow to suggest how to break down a large task so that I can plan complex assignments.	Low	8

5.2 Initial Acceptance Criteria

- US-01 Account Registration
 - A user can enter a valid name, email address, and password.
 - Duplicate email addresses are rejected.
 - Invalid required fields produce an understandable error message.
 - Passwords are not stored as plain text.
 - A successfully created account can later be used to log in.
- US-02 Login/Logout
 - A registered user can log in using a valid email and password.
 - Invalid credentials are rejected with an understandable error message.
 - A logged-in user can successfully log out.
- US-03 Course Management
 - An authenticated user can create a course.
 - The user can edit or delete their own course.
 - Created courses are available when assigning a task.
- US-04 Create Academic Task
 - An authenticated user can create a task.
 - A task must contain a title and valid due date.
 - The user can associate the task with one of the user's courses.
 - The user can select a task type and user-defined priority level.

- The user may provide estimated work hours and current progress.
 - After saving, the task appears in the user's task list or dashboard.
- US-05 Edit/Delete Task
 - A user can edit the details of their own academic task.
 - A user can delete their own academic task.
 - Changes are reflected on the dashboard after saving.
- US-06 Dashboard
 - Only the authenticated user's academic tasks are displayed.
 - Active tasks are visible without navigating to individual course pages.
 - Completed tasks are visually distinguishable from active tasks.
 - Important task information such as title, course, deadline, user-defined priority, and progress is visible.
- US-07 Progress
 - A user can enter a task progress value from 0-100%.
 - A user can set the task status to Not Started, In Progress, or Completed.
 - When a task is marked Completed, its progress is set to 100%.

5.3 Functional Requirements

- FR-01: Allow users to create accounts.
- FR-02: Authenticate registered users.
- FR-03: Allow users to create, update, and delete courses.
- FR-04: Allow users to create, update, and delete academic tasks.
- FR-05: Associate tasks with courses.
- FR-06: Store task deadlines, user-defined priorities, estimated work hours, progress, and status.
- FR-07: Display active academic tasks on a dashboard.
- FR-08: Allow users to track task progress from 0-100% and set the task status as Not Started, In Progress, or Completed.
- FR-09: Prevent users from accessing another user's private academic data.
- FR-10: Provide search and filtering capabilities in a later iteration.
- FR-11: Calculate and display Task Attention Levels in a later iteration.

5.4 Nonfunctional Requirements

- Usability: The primary workflow should be understandable without specialized training.
- Performance: Normal dashboard and task operations should respond quickly under expected classroom-project usage.
- Reliability: Saved academic information should persist across sessions.
- Security: Passwords must be securely hashed, protected routes must require authentication, and users must not be able to access another user's private data.
- Privacy: Academic data will be private to the authenticated account unless future sharing functionality is explicitly implemented.
- Accessibility: The interface should use readable text, meaningful labels, keyboard-accessible controls where practical, and sufficient visual contrast.
- Maintainability: The application should use clear module boundaries, documented APIs, consistent coding conventions, and automated tests.
- Portability: The application should be reproducible on an independent development computer using documented setup instructions.

6. Initial Design & Architecture

6.1 Proposed Technology Stack

- Frontend: React
- Backend: Node.js with Express
- Database: PostgreSQL
- API Style: REST/JSON
- Authentication: Secure password hashing and token- or session-based authentication, with the final mechanism to be confirmed during design
- Version Control: Git and GitHub
- Testing: Automated frontend/backend testing tools appropriate to the chosen implementation
- CI/CD: GitHub Actions or an equivalent service
- Containerization: Docker
- Deployment: A publicly accessible cloud-hosting platform suitable for the final architecture

6.2 Proposed Architecture

StudyFlow will initially use a three-tier client-server architecture.

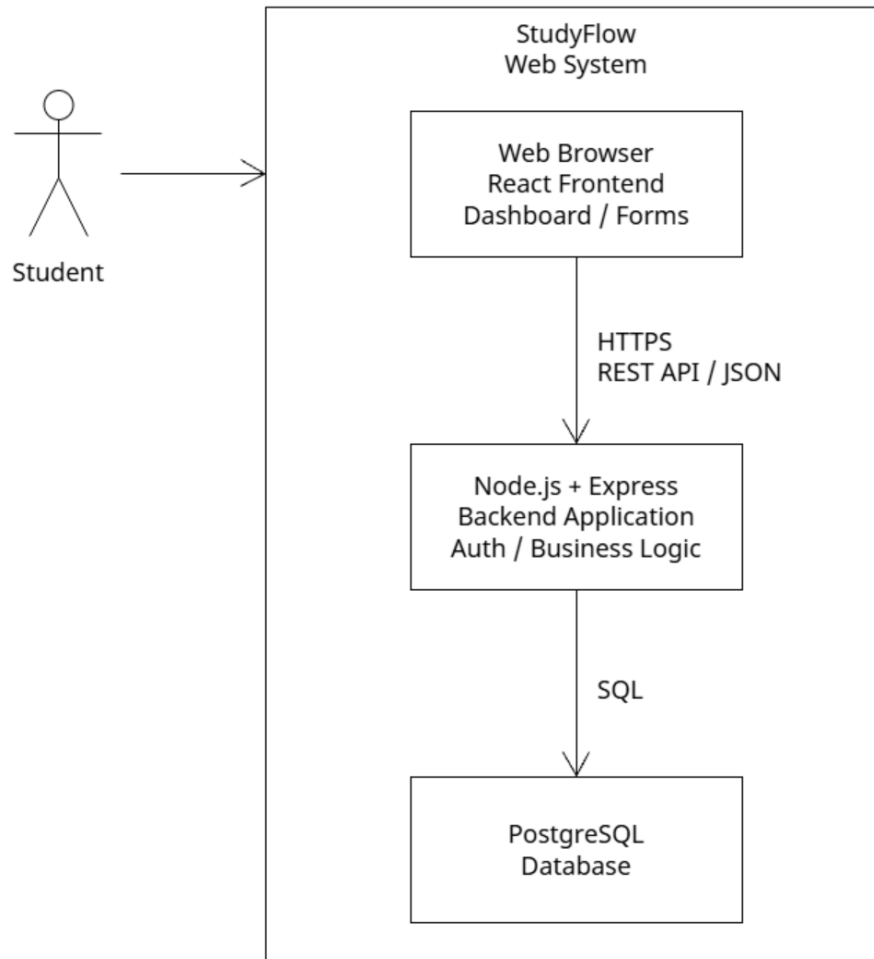


Figure 1. StudyFlow High-Level System Architecture

6.3 Technology Choice Rationale

React provides a component-based approach suitable for an interactive student dashboard. Node.js and Express support straightforward development of REST APIs and allow the team to use JavaScript across much of the application stack. PostgreSQL is appropriate because StudyFlow contains structured relational data such as users, courses, and academic tasks.

The proposed stack is also suitable for automated testing, containerization, CI/CD, and deployment, which are required in later project milestones.

7. Data & External Resources

7.1 Main Data Entities

The initial StudyFlow data model will contain three primary entities.

- User
Stores account information.
Possible attributes:
 - user_id
 - name
 - email
 - password_hash
 - created_at
- Course
Represents a course belonging to a user.
Possible attributes:
 - course_id
 - user_id
 - course_name
 - course_code
 - semester
- AcademicTask
Represents an assignment, exam, project, or other academic responsibility.
Possible attributes:
 - task_id
 - user_id
 - course_id
 - title
 - description
 - task_type
 - due_date
 - priority
 - estimated_hours
 - progress
 - status

- created_at
- updated_at

7.2 External Resources

The MVP is intentionally designed to minimize dependence on third-party services. Core functionality should operate using the team's own frontend, backend, and database.

Potential future integrations may include calendar APIs, notification services, or an AI API, but these services are not required for the initial MVP.

Any external service introduced later will be evaluated for cost, API limits, privacy, licensing, and availability before adoption.

The MVP does not require external datasets, specialized devices, or third-party APIs. The application will rely on the frameworks and libraries identified in the proposed technology stack, while cloud hosting will be introduced during the deployment milestone.

8. Agile Development Process

8.1 Sprint Length

The team plans to use one-week development sprints. Each sprint will begin with selection of a small set of prioritized backlog items and end with review of completed work and adjustment of upcoming priorities.

8.2 Backlog and Issue Tracking

The prioritized product backlog will be maintained in the shared GitHub repository. GitHub Issues may be used to track implementation tasks, defects, and milestone-related work.

8.3 Branching and Pull Request Strategy

The main branch will represent stable integrated project work. Significant features will be developed on short-lived feature branches, such as feature/authentication, feature/course-management, or feature/task-dashboard, and merged into main through pull requests.

Example:

main

└ feature/authentication

└ feature/course-management

└ feature/task-dashboard

Changes will normally be merged through pull requests. The other team member should review meaningful pull requests when practical so that both members remain familiar with the codebase and peer-review evidence is preserved.

8.4 Definition of Done

A backlog item is considered Done when:

- The agreed acceptance criteria are satisfied.
- The implementation runs successfully.
- Relevant automated tests have been added or updated when applicable.
- Existing tests continue to pass.
- The code has been reviewed when appropriate.
- Documentation is updated if setup, behavior, or interfaces changed.
- No known critical defects prevent the feature from being demonstrated.
- The completed work has been integrated into the appropriate shared branch.

8.5 Communication

The two team members will communicate regularly through GroupMe, use GitHub for version control and project tracking, and use a shared Google folder for collaborative project documents.

9. Milestone Plan

9.1 Development Approach

Each development milestone (Milestones 1, 2, and 3) will contain a runnable version of StudyFlow. Each version will build incrementally on the functionality completed in the previous milestone.

9.2 Development Schedule

Milestone 1 will focus on implementing the core end-to-end MVP. Milestone 2 will expand the working application with Task Attention Level logic, testing, security, and quality improvements. Milestone 3 will focus on deployment, CI/CD, containerization, final UI/UX improvements, documentation, and release preparation. Development within each milestone will follow the team's one-week sprint cycle and the deadlines established in the course schedule.

9.3 Milestone 1 – Build It

Milestone 1 will establish the working end-to-end StudyFlow MVP.

Planned functionality:

- User registration and authentication
- Course CRUD operations
- Academic task CRUD operations
- Due dates and user-defined priorities
- Estimated work hours
- Progress/status tracking
- Basic dashboard
- Persistent database storage
- Setup and verification instructions

A student should be able to register, log in, create a course, create an academic task, and view the saved task on the dashboard.

9.4 Milestone 2 – Test and Improve It

Milestone 2 will expand the working application and focus on quality and security.

Planned additions include:

- Task Attention Level calculation and display
- Search and filtering
- Improved progress visualization
- Unit tests
- Integration tests
- Coverage reporting

- Peer code-review evidence
- Threat modeling
- Authentication/authorization security testing
- Input validation
- Review of relevant OWASP Top 10 risks
- Security-fix evidence

9.5 Milestone 3 – Deploy It

Milestone 3 will complete and deploy StudyFlow as the final working product.

Planned work includes:

- UI/UX polish
- Responsive improvements
- Containerized local environment
- Automated linting and testing
- CI pipeline
- CD pipeline
- Public deployment
- Metrics reporting
- Final security and quality improvements
- Documented AI-assisted development workflow
- Maintenance plan
- Final README
- Final live demonstration
- milestone-3 tag and v1.0 release

10. Team Responsibilities

- Eunjoo Jung
Eunjoo will initially focus on backend development, database design, business logic, Task Attention Level logic, API integration, testing, and security. She will also participate in frontend integration, code review, documentation, deployment, and presentation preparation.
- Errin James
Errin will initially focus on frontend development, UI/UX design, dashboard presentation, responsive layouts, and visual representations of

task priority and progress. She will also participate in backend integration, testing, code review, documentation, deployment, and presentation preparation.

- **Shared Responsibility Statement**
Although each member will have an initial area of emphasis, responsibilities will not be strictly divided into isolated frontend and backend roles. Both members will contribute to coding, testing, documentation, review, deployment, and major design decisions so that knowledge and responsibility are shared across the project.

11. Risks & Mitigation

Risk	Potential Impact	Mitigation
Scope becomes too large	Project may not be completed on schedule	Keep AI, calendar integration, and advanced analytics outside the MVP
Task Attention Level logic becomes overly complex	Core application development could be delayed	Begin with a simple transparent rule-based model
Authentication/security defects	Private academic data could be exposed	Use established security practices, authorization checks, automated tests, and OWASP review
Uneven team workload	One member may become overloaded	Review backlog and responsibilities weekly
Integration conflict	Frontend/backend work may not combine cleanly	Define API contracts early and use frequent integration
Deployment problems	Final application may not be publicly accessible	Test deployment before Milestone 3 and maintain reproducible local setup
External-service dependency	API cost or availability could block features	Keep external APIs optional rather than required for core functionality

Schedule pressure	Quality/testing may be postponed	Deliver working software incrementally instead of saving integration until the end
-------------------	----------------------------------	--

12. Success Criteria

- Users should be able to create an account with email and password to keep their information secure.
- Users should be able to log in with those credentials successfully to access their own courses and academic tasks saved on their account.
- Users should be able to create tasks with a title, type, due date, estimated work hours, user-defined priority, and associated course.
- Users should be able to create courses to sort their tasks into.
- Users should be able to view their tasks and related details, including title, due date, user-defined priority, course, estimated work hours, type, progress, and status, after logging in.
- Users should be able to update task progress and mark tasks as completed.
- Users should be able to clearly see the Task Attention Level of active tasks, determined using factors such as due date, estimated work hours, user-defined priority, and progress.
- Users should not be able to access or modify other users' private academic data.
- StudyFlow should be runnable using the documented setup instructions and should be available through the required deployment method by the final milestone.
- Important application behavior should be verified through automated testing.

References

- Project Milestones. Course project requirements and milestone guidelines provided by the instructor. Used to define the required proposal structure, agile development expectations, MVP requirements, testing and security requirements, deployment expectations, and contribution tracking.
- React Documentation. Meta Platforms, Inc. *React Documentation – Quick Start*. Used as a reference for the proposed component-based frontend architecture. <https://react.dev/learn>
- Express.js Documentation. OpenJS Foundation and Express contributors. *Express.js Documentation*. Used as a reference for the proposed Node.js and Express backend architecture. <https://expressjs.com/>
- PostgreSQL Global Development Group. *PostgreSQL 18 Documentation*. Used as a reference for the proposed relational database architecture and structured data storage. <https://www.postgresql.org/docs/current/index.html>
- OWASP Foundation. *OWASP Top 10:2025*. Used as a reference for identifying common web-application security risks that will be reviewed and addressed during later milestones. <https://owasp.org/Top10/2025/>
- GitHub Docs. *About Pull Requests*. Used as a reference for the proposed feature-branch, pull-request, peer-review, and collaboration workflow. <https://docs.github.com/en/pull-requests/get-started/about-pull-requests>

Milestone 0 deliverables

1. Project proposal PDF

See cover page and pages 1 - 18

2. Shared GitHub repository link

<https://github.com/hihi0305/studyflow>

3. Initial README and prioritized backlog

- a. README

StudyFlow

StudyFlow is a web-based academic planning application designed to help college students organize courses, assignments, exams, and projects while identifying academic tasks that may require immediate attention.

Problem

College students often manage multiple deadlines across several courses. Traditional calendars and task lists can show what needs to be completed, but they provide limited support for evaluating workload, progress, and which task should receive attention first.

Proposed Solution

StudyFlow combines course and academic task management with deadline, estimated work hours, user-defined priority, and progress information in a centralized dashboard.

The application will help students organize their academic responsibilities and identify tasks that may require immediate attention. In a later milestone, StudyFlow will calculate and display Task Attention Levels based on factors such as due date, estimated work hours, user-defined priority, and progress.

Planned Core Features

Core MVP Features

- User registration and authentication
- Course creation, editing, and deletion
- Academic task creation, editing, and deletion
- Task type selection
- Due-date tracking
- User-defined priority
- Estimated work hours
- Task progress and status tracking
- Centralized academic dashboard
- Persistent database storage
- User-specific data access

Planned Later Features

- Task Attention Level calculation and display
- Search and filtering
- Improved progress visualization
- Workload-based task recommendations
- Calendar or timeline view
- Academic workload analytics
- Reminder notifications
- AI-assisted task decomposition

Proposed Technology Stack

- Frontend: React
- Backend: Node.js with Express
- Database: PostgreSQL
- API Style: REST/JSON
- Authentication: Secure password hashing and token- or session-based authentication, with the final mechanism to be confirmed during design
- Version Control: Git and GitHub
- Testing: Automated frontend/backend testing tools appropriate to the chosen implementation
- CI/CD: GitHub Actions or an equivalent service

- Containerization: Docker
- Deployment: A publicly accessible cloud-hosting platform suitable for the final architecture

Development Approach

StudyFlow will be developed incrementally across the course milestones. Each development milestone will contain a runnable version of the application and will build on the functionality completed in the previous milestone.

- **Milestone 0:** Planning, requirements, architecture, and prioritized backlog
- **Milestone 1:** Working end-to-end MVP
- **Milestone 2:** Task Attention Level logic, testing, security, and quality improvements
- **Milestone 3:** Deployment, CI/CD, containerization, documentation, and final release

Development will follow one-week sprints, with prioritized backlog items selected and reviewed throughout the project.

Definition of Done

A backlog item is considered Done when:

- [] The agreed acceptance criteria are satisfied.
- [] The implementation runs successfully.
- [] Relevant automated tests have been added or updated when applicable.
- [] Existing tests continue to pass.
- [] The code has been reviewed when appropriate.
- [] Documentation is updated if setup, behavior, or interfaces changed.
- [] No known critical defects prevent the feature from being demonstrated.
- [] The completed work has been integrated into the appropriate shared branch.

Development Process

Backlog and Issue Tracking

The prioritized product backlog will be maintained in the shared GitHub repository. GitHub Issues may be used to track implementation tasks, defects, and milestone-related work.

See docs/BACKLOG.md for the current prioritized product backlog.

Branching and Pull Request Strategy

The `main` branch will represent stable integrated project work.

Significant features will be developed on short-lived feature branches, such as:

```

``text
main
├── feature/authentication
├── feature/course-management
└── feature/task-dashboard

```

b. BACKLOG

StudyFlow Product Backlog

This backlog contains the initial prioritized user stories for StudyFlow.

Definition of Done: see [README.md](../README.md).

ID	User Story	Priority	Initial Estimate	Status
US-01	As a new user, I want to create an account so that I can securely access my academic information.	High	3	Backlog
US-02	As a registered user, I want to log in and log out securely so that my academic data remains private.	High	5	Backlog
US-03	As a student, I want to create and manage courses so that I can organize tasks by course.	High	3	Backlog
US-04	As a student, I want to create academic tasks so that I can track assignments, exams, and projects.	High	5	Backlog

| US-05 | As a student, I want to edit and delete tasks so that my academic information remains accurate. | High | 3 | Backlog |

| US-06 | As a student, I want to view my active tasks on a dashboard so that I can quickly understand my workload. | High | 5 | Backlog |

| US-07 | As a student, I want to record task progress so that I know how much work remains. | High | 3 | Backlog |

| US-08 | As a student, I want StudyFlow to assign Task Attention Levels to my active tasks so that I can decide what to work on first. | Medium | 5 | Backlog |

| US-09 | As a student, I want to search and filter tasks so that I can quickly find relevant academic work. | Medium | 3 | Backlog |

| US-10 | As a student, I want to view workload information so that I can better plan upcoming study time. | Medium | 5 | Backlog |

| US-11 | As a student, I want a calendar or timeline view so that I can understand upcoming deadlines visually. | Low | 5 | Backlog |

| US-12 | As a student, I want StudyFlow to suggest how to break down a large task so that I can plan complex assignments. | Low | 8 | Backlog |

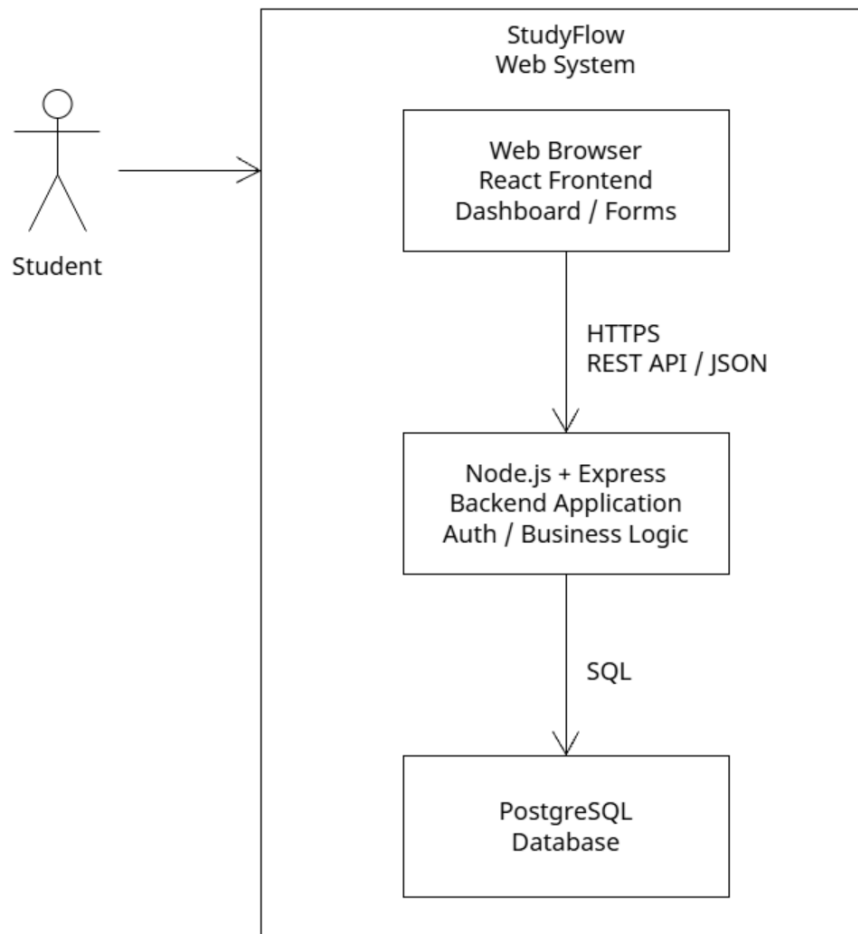
Priority Level Key

- **High:** Required for the core MVP and should be implemented first.
- **Medium:** Important features planned after the core MVP is stable; they may be added in a later milestone if time allows.
- **Low:** Optional enhancements considered after High- and Medium-priority work is complete.

Status Definitions

- **Backlog:** Not yet started.
- **In Progress:** Currently being implemented.
- **In Review:** Implementation is complete and awaiting review or verification.
 - **Done:** Meets the project's Definition of Done and has been integrated into the appropriate shared branch.

4. Initial architecture or system-context diagram



5. One- or two-sentence contribution statement for every member

Milestone 0 Contribution Statements

Eunjoo Jung

Eunjoo contributed to most sections of the Milestone 0 proposal, including the executive summary, MVP scope, initial requirements, architecture, data and external resources, agile process, milestone plan, team responsibilities, risks and mitigation, and references. She also performed the final document review and revisions for clarity, consistency, formatting, terminology, and completeness, managed the GitHub repository and push process, and shared responsibility for the README and prioritized backlog.

Errin James

Errin contributed to the Target Users & Personas, Proposed Application, and Success Criteria sections of the Milestone 0 proposal and revised those sections based on team feedback. She also shared responsibility for developing the README and prioritized backlog.